SafeCity

Complete Technology Stack Reference

Smart City Incident Management and Civic Response Platform

The Bombay Salesian Society — SDP Project

Document Version 1.0

October 2026

Table of Contents

1 Frontend — What Users See

2 Backend — The Server Logic

3 Database & Caching

4 Asynchronous Processing

5 File Storage

6 Containerization & DevOps

7 Testing

8 Monitoring & Observability

9 Deployment & Infrastructure

10 Documentation & API Specification

11 System Architecture Overview

1. Frontend — What Users See

The frontend is the web application users interact with in their browser. It is a single-page application built with React and served by Vite during development or Nginx in production.

Technology	What It Does
React 18	The main UI library. Builds every web page (dashboard, map, forms) out of reusable components.
TypeScript	JavaScript with static type-checking. Catches bugs at compile time before the code ever runs.
Vite	Development server and build tool. Compiles and hot-reloads the frontend almost instantly.
Tailwind CSS	Utility-first CSS framework. Styling is done by composing class names directly in markup.
Radix UI	Accessible UI primitives. Provides dialogs, dropdowns, tooltips, and tabs that work with keyboards and screen readers.
React Router v6	Client-side routing. Maps URL paths such as /login, /dashboard, and /incidents to page components.
TanStack Query	Server-state management. Fetches API data, caches responses, and automatically refetches every 30 seconds.
Zustand	Client-state management. Stores the logged-in user session and theme preference (dark / light mode) in memory.
Axios	HTTP client. Sends all API requests to the Django backend (for example GET /api/v1/incidents/).
React Hook Form	Form state management. Handles multi-step report wizard forms with minimal re-renders.
Zod	Schema validation. Defines type-safe validation rules for every form field.
Leaflet + React Leaflet	Interactive maps. Displays incidents as markers on an OpenStreetMap base layer with clustering support.
Recharts	Charting library. Renders analytics graphs (bar, line, pie) on authority dashboards.
Sonner	Toast notifications. Shows transient success, error, and warning messages after user actions.
Lucide React	Icon set. Provides the vector icons used throughout the interface.
Class Variance Authority	Component variant system. Manages button sizes, colours, and variants declaratively.
Service Worker / PWA	Offline support. Caches assets so the app remains partially functional without connectivity.

2. Backend — The Server Logic

The backend is a Python Django application that exposes a REST API, enforces authentication and role-based access control, and contains all business logic for incident management.

Technology	What It Does
Django 5	Python web framework. Handles URL routing, the ORM (database queries), sessions, security middleware, and the admin panel.
Django REST Framework	API toolkit. Builds JSON REST endpoints with serializers, pagination, and permission classes.
SimpleJWT	JWT authentication. Issues short-lived access tokens (15 minutes) and rotating refresh tokens (7 days) with blacklist support.
django-cors-headers	CORS control. Permits the frontend origin (port 5174) to make cross-origin requests to the backend (port 18081).
django-filter	Query filtering. Enables query-string filters such as ?status=open&severity=high on list endpoints.
drf-spectacular	OpenAPI schema generator. Produces interactive Swagger documentation at /api/schema/swagger/.
django-environ	Environment configuration. Reads .env variables (secret key, database password) into Django settings.
Argon2 (argon2-cffi)	Password hashing. Securely stores user passwords using the Argon2 algorithm, resistant to brute-force attacks.
PyJWT + cryptography	Token signing and encryption. Verifies JWT tokens and implements TOTP-based two-factor authentication.
google-auth	Google OAuth. Verifies Google ID tokens for the Sign in with Google login and signup flow.
Pillow	Image processing. Validates uploaded incident photos, checks dimensions, and strips EXIF metadata for privacy.
boto3	AWS S3 SDK. Uploads and retrieves media files from MinIO, the S3-compatible object store.
Gunicorn	WSGI production server. Runs Django with multiple worker processes to handle concurrent requests.
Daphne	ASGI server. Runs Django with WebSocket support required for real-time notification delivery.
WhiteNoise	Static file server. Serves compiled CSS, JavaScript, and images efficiently from the Django process.
Requests	HTTP client. Calls external services such as the Nominatim geocoding API for address lookup.
Channels + channels-redis	WebSocket framework. Fans out real-time notifications to connected browser clients via Redis.

Technology	What It Does
psycopg 3	PostgreSQL driver. The Python library Django uses to communicate with the database.

3. Database & Caching

Technology	What It Does
PostgreSQL 16	Primary relational database. Stores all application data (users, incidents, departments, audit log) in normalised tables. Supports full-text search, JSONB columns, and GIN indexes for performance.
Redis 7	In-memory data store. Serves two roles: a fast cache for frequently accessed data, and the message broker / result backend for Celery task queues.

4. Asynchronous Processing

Technology	What It Does
Celery	Distributed task queue. Runs slow or scheduled jobs (sending emails, computing daily analytics) in background workers so HTTP responses are never blocked.
django-celery-beat	Scheduler. Triggers recurring tasks on a timetable, such as the SLA breach sweep that runs every five minutes.
Django Channels	WebSocket support. Enables bidirectional communication so new notifications appear in the browser instantly without polling.

5. File Storage

Technology	What It Does
MinIO	Self-hosted object storage. An open-source, S3-compatible replacement for Amazon S3 that stores all incident photo and video evidence locally at zero cost.

6. Containerization & DevOps

Technology	What It Does
Docker	Containerization platform. Packages each service (backend, database, cache, frontend) into an isolated, reproducible container that runs identically on any machine.
Docker Compose	Multi-container orchestrator. A single command (docker compose up -d) starts all seven services with correct networking and volumes.
Docker Compose overlay files	Configuration layers. docker-compose.tunnel.yml and docker-compose.prod.yml extend the base file for tunnel and production use without duplicating configuration.
GitHub Actions	Continuous integration and delivery. Automatically runs linting, tests, and Docker builds on every push and pull request.
Ruff	Python linter and formatter. Checks Python source for style violations and common bugs, and auto-fixes many of them.
ESLint	JavaScript / TypeScript linter. Enforces code quality rules across the React codebase.
Prettier	Opinionated code formatter. Automatically formats source files to a consistent style.
Make	Command runner. Provides short aliases (make docker-up, make test, make lint) for long command sequences.

7. Testing

Technology	What It Does
pytest	Python test runner. Executes the backend test suite (227 tests) covering accounts, incidents, workflow, RBAC, and analytics.
pytest-cov	Coverage measurement. Reports the percentage of source lines exercised by tests and enforces an 80 percent gate in CI.
Vitest	Frontend test runner. Runs unit and component tests for the React application (15 tests) using the Vite ecosystem.
Testing Library (React)	Component testing utilities. Simulates user interactions such as clicks, typing, and form submission on rendered components.
Playwright	End-to-end browser testing. Drives a real Chromium browser through complete user journeys (login, report submission, admin queue).
axe-core for Playwright	Accessibility testing. Scans rendered pages for WCAG 2 A / AA violations to ensure screen-reader compatibility.
jsdom	Headless DOM implementation. Allows React component tests to run inside Node.js without launching a real browser.
TypeScript Compiler (tsc)	Type checking. The build step runs tsc -b to verify there are no type errors before bundling.

8. Monitoring & Observability

Technology	What It Does
Prometheus	Metrics collection server. Scrapes the /api/metrics/ endpoint at regular intervals and stores time-series data on request counts, latencies, and error rates.
Grafana	Metrics dashboards. Queries Prometheus and visualises the results as graphs and alert panels for operators.
Django health endpoints	Health probes. /api/health/ confirms the process is alive; /api/readiness/ verifies database, cache, and storage connectivity.
Structured logging	Machine-readable logs. Application events are emitted in a consistent format that is easy to search and parse.

9. Deployment & Infrastructure

Technology	What It Does
Cloudflare Quick Tunnels	Free public exposure. Exposes the local stack to the internet through an ephemeral trycloudflare.com URL with no account and no credit card.
cloudflared	Tunnel client. The command-line tool that creates and maintains the secure tunnel between Cloudflare edge and the local ports.
Terraform	Infrastructure as Code. Declares cloud resources in version-controlled .tf files; fmt and validate pass locally (apply skipped due to no-cloud-account constraint).
Kubernetes (Kind)	Container orchestration. A local Kubernetes cluster used to validate deployment manifests without requiring a cloud provider.
Helm	Kubernetes package manager. Deploys the application to a cluster using templated, environment-specific values files.
Nginx	Reverse proxy and web server. Serves compiled frontend assets and proxies /api/ and /ws/ requests to the Django backend in production.

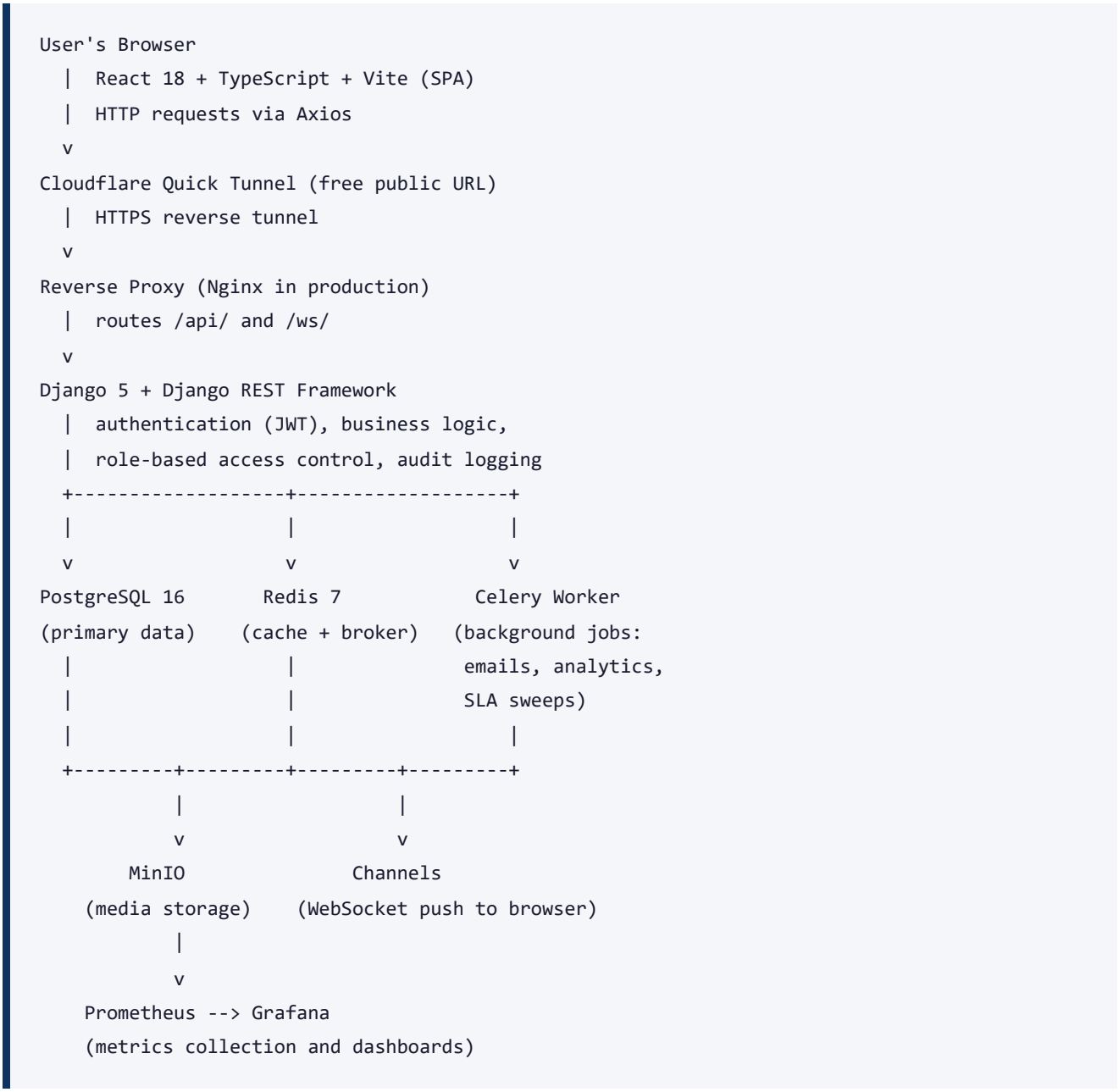
> **Note:** No AWS or paid cloud services are used. The entire stack runs at zero cost using Docker Desktop, local Kubernetes (Kind), and Cloudflare quick tunnels.

10. Documentation & API Specification

Technology	What It Does
Swagger UI	Interactive API documentation. Allows developers to browse every endpoint and execute test calls directly from the browser.
OpenAPI (via drf-spectacular)	Machine-readable API specification. Generates docs/api-schema.yaml describing all request and response shapes.
Markdown	Documentation format. All project documents (README, architecture, deployment guide, audit reports) are written in .md files tracked in the repository.

11. System Architecture Overview

The following diagram shows how a user request flows through every layer of the stack:



All seven services are orchestrated by Docker Compose. Every push to GitHub triggers the CI pipeline (Ruff lint, ESLint, pytest with 80 percent coverage gate, Vitest, Playwright end-to-end tests, and Docker image builds).

Service Inventory

Service	Container	Port	Purpose
db	postgres:16-alpine	5432	Primary relational database
redis	redis:7-alpine	6379	Cache and message broker
minio	minio/minio	9000 / 9001	Object storage for media files
backend	Django + Daphne	18081 (host)	REST API and WebSocket server
celery	Celery worker	-	Background task execution
celery-beat	Celery beat	-	Scheduled / periodic tasks
frontend	Vite dev server	5174 (host)	React application

Environment Summary

Category	Tools Used
Languages	Python 3.12, TypeScript 5.5, HTML / CSS, SQL, HCL (Terraform)
Frameworks	Django 5, Django REST Framework, React 18, Vite
Data	PostgreSQL 16, Redis 7, MinIO (S3-compatible)
Async	Celery, django-celery-beat, Django Channels
Testing	pytest, Vitest, Playwright, axe-core, Ruff, ESLint, Prettier
Infrastructure	Docker, Docker Compose, Kubernetes (Kind), Helm, Terraform
CI/CD	GitHub Actions
Monitoring	Prometheus, Grafana, Django health endpoints
Deployment	Cloudflare Quick Tunnels (free), Nginx